CSE 598 008
EECS 498 003
Lab 9

Tianyiming (Henry) Jing

October 30, 2026

Happy Halloween!

- You did better on the midterm than previous semesters. Well done!
- Donuts are here, help yourself

Thank You

- Thank you for the midterm feedback!
- I read every comment, and it made my week

Reminder

- Project 1 (distributed lock service) due November 9
- Midterm regrade requests open until October 31

Today

- World-visible events, from Thursday's lecture
- Exercise: a lock service spec with events
- Preview of next week: refinement
- Questions about Project 1? Ask anytime

World-Visible Events

```
datatype Event =
    | Acquire(clientId: nat)
    | Release(clientId: nat)
    | NoOp

ghost predicate Next(c: Constants,
    v: Variables, v': Variables,
    evt: Event)
```

- The outside world sees only events, not the state inside the system
- A behavior is a sequence of events; correctness is judged on behaviors
- A spec state machine describes all correct behaviors: each step is labeled by an event
- `NoOp` labels a step that the outside world does not see

Exercise

- Starter code: `lab09.dfy` on the course GitHub
  - `UtilitiesLibrary.dfy` is in the same folder
- Part 1: write the spec of the lock service, with events
- Part 2: which behaviors from lecture does your spec accept? Prove it
  - Accepted: show a sequence of states that works
  - Not accepted: show that no sequence of states works

```
Behavior1: [Acquire(1), Acquire(1), Release(1), Release(1)]
Behavior2: [Release(2), Acquire(1), Release(1)]
Behavior3: [Acquire(1), Release(1), Acquire(2)]
```

Preview - Refinement

- Part 3: a lock server implements the lock service (given)
  - `RequestLock`: a client asks for the lock (`NoOp`)
  - `GrantLock`: the server gives the lock to a waiting client (`Acquire`)
  - `ReturnLock`: the client gives the lock back (`Release`)
- Write the abstraction function: which spec state does each lock server state stand for?
- Dafny checks it for you, no proof needed:
  - Each initial state stands for an initial spec state
  - Each step is a spec step with the same event, or a stutter step: the event is NoOp and the spec state does not change
- Next week in lecture: the full refinement proof

Extra - The Lottery Paradox

- You parked your car on the street this morning
  - Do you know your car is still there?
- You bought a Powerball ticket. The draw was last night, and you have not checked
  - Do you know your ticket lost?

Extra - The Lottery Paradox

- Most people: yes to the car, no to the ticket
  - Car stolen today: roughly 1 in 150,000 (about 660,000 US cars stolen in 2025)
  - Ticket wins the jackpot: 1 in 292,201,338
- The car is about 2,000 times more likely to be gone, yet we say we know it is there
- For anything you say you know, a big enough lottery is even less likely to win, yet you don’t know your ticket lost
- So knowing something is not just about how likely it is
- One answer: a stolen car would need an explanation; a winning ticket would not, since some ticket wins
- Still debated today

https://en.wikipedia.org/wiki/Lottery_paradox